

TP 3 : Construire un agent d'étude avec outils

Module : IA Générative

TP guidé, simple à exécuter, avec forte valeur pratique

Nom et prénom		Groupe	
Date		Durée	1h30

1. Pourquoi ce TP ?

Dans ce TP, on ne veut pas seulement utiliser un chatbot.
On veut apprendre une compétence plus utile :

transformer un besoin concret en mini agent capable d'utiliser des outils.

Valeur ajoutée perçue pour l'étudiant :

- savoir construire un agent simple en Python ;
- comprendre clairement la différence entre **chatbot** et **agent** ;
- apprendre à définir des **tools** ;
- comprendre le **tool calling** ;
- repartir avec une base réutilisable dans d'autres projets.

2. Scénario choisi

Le TP propose de construire un **agent d'étude**.
Cet agent pourra :

- calculer une moyenne à partir de notes ;
- chercher une information dans des notes locales ;
- générer un petit plan de révision.

Pourquoi ce choix est pertinent ?

- il est simple à exécuter ;
- il est utile pour un étudiant ;
- il montre visiblement l'intérêt des outils ;
- il ouvre vers des projets plus avancés.

3. Objectifs du TP

À la fin de ce TP, l'étudiant doit être capable de :

- expliquer ce qu'est un agent IA ;
- expliquer le rôle des outils dans un agent ;
- utiliser **Groq** comme modèle de raisonnement ;
- définir des outils Python simples ;
- relier ces outils à un modèle ;
- tester un mini agent interactif ;
- proposer une extension sous forme d'un quatrième outil.

4. Rappel de cours

Un chatbot : répond en texte à une consigne.

Un agent :

- reçoit un objectif ;
- choisit une action ;
- peut utiliser un outil ;
- observe le résultat ;
- produit une réponse plus utile.

Idée clé du TP : ici, l'agent n'est pas intelligent parce qu'il parle seulement. Il devient plus utile parce qu'il peut **agir sur plusieurs types de besoins** via des outils.

5. Pré-requis

- Python 3.10 ou 3.11 ;
- un terminal ;
- une clé API Groq ;
- le fichier `.env` avec :

```
1 GROQ_API_KEY=votre_cle_api_ici
```

6. Installation

Sous Windows :

```
1 python -m venv .venv
2 .venv\Scripts\activate
3 pip install --upgrade pip
4 pip install langchain langchain-core langchain-groq python-dotenv
```

Sous Linux / macOS :

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install --upgrade pip
4 pip install langchain langchain-core langchain-groq python-dotenv
```

7. Fichiers utilisés

Le TP repose sur deux fichiers :

- `agent_etudiant_tp.py`
- `notes_agent_tp.txt`

Rôle du fichier `notes_agent_tp.txt` : il représente une mini base locale que l'agent peut consulter comme outil.

Exemples de lignes présentes dans ce fichier :

- “Le RAG combine la recherche d’information et la génération de texte.”
- “Un agent IA peut choisir un outil selon l’objectif à atteindre.”
- “PromptTemplate permet de construire un prompt dynamique avec des variables.”

8. Code complet de l'agent

Enregistrer le code suivant dans `agent_etudiant_tp.py`.

Listing 1 – Code complet du mini agent d'étude avec outils

```

1 import json
2 import os
3 from pathlib import Path
4
5 from dotenv import load_dotenv
6 from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage
7 from langchain_core.tools import tool
8 from langchain_groq import ChatGroq
9
10
11 NOTES_PATH = Path("notes_agent_tp.txt")
12 MODEL_NAME = "llama-3.3-70b-versatile"
13
14
15 @tool
16 def calculer_moyenne(notes_json: str) -> str:
17     """Calcule la moyenne, la note minimale et la note maximale a partir d'une liste JSON de notes."""
18     try:
19         notes = json.loads(notes_json)
20         if not isinstance(notes, list) or not notes:
21             return "Erreur : fournissez une liste JSON non vide, par exemple [12, 15, 9]."
22
23         notes = [float(note) for note in notes]
24         moyenne = sum(notes) / len(notes)
25         minimum = min(notes)
26         maximum = max(notes)
27
28         return (
29             f"Moyenne : {moyenne:.2f}\n"
30             f"Minimum : {minimum:.2f}\n"
31             f"Maximum : {maximum:.2f}"
32         )
33     except Exception as exc:
34         return f"Erreur pendant le calcul : {exc}"
35
36
37 @tool
38 def chercher_dans_les_notes(mot_cle: str) -> str:
39     """Cherche un mot-cle simple dans le fichier de notes local de l'etudiant et retourne les lignes pertinentes."""
40     if not NOTES_PATH.exists():
41         return "Erreur : le fichier notes_agent_tp.txt est introuvable."
42
43     lines = NOTES_PATH.read_text(encoding="utf-8").splitlines()
44     mot_cle = mot_cle.strip().lower()
45
46     if not mot_cle:
47         return "Erreur : fournissez un mot-cle non vide."
48
49     matches = [line for line in lines if mot_cle in line.lower()]
50
51     if not matches:
52         return f"Aucune ligne trouvee pour le mot-cle : {mot_cle}"
53
54     return "\n".join(matches[:5])
55
56
57 @tool
58 def generer_plan_revision(sujet: str) -> str:
59     """Genere un mini plan de revision en 5 points a partir d'un sujet simple."""
60     sujet = sujet.strip()
61     if not sujet:
62         return "Erreur : fournissez un sujet."
63
64     return (
65         f"Plan de revision pour : {sujet}\n"
66         "1. Relire les definitions et notions de base.\n"
67         "2. Identifier 5 mots-cles essentiels.\n"
68         "3. Refaire un exemple concret ou un mini exercice.\n"

```

```
69     "4. Resumer le sujet en 5 lignes maximum.\n"
70     "5. Se tester avec 3 questions de revision."
71 )
72
73
74 TOOLS = [calculer_moyenne, chercher_dans_les_notes, generer_plan_revision]
75 TOOLS_BY_NAME = {tool_.name: tool_ for tool_ in TOOLS}
76
77
78 def run_agent(question: str) -> str:
79     load_dotenv()
80
81     api_key = os.getenv("GROQ_API_KEY")
82     if not api_key:
83         raise EnvironmentError(
84             "La variable GROQ_API_KEY est absente. Ajoutez-la dans un fichier .env."
85         )
86
87     llm = ChatGroq(
88         model=MODEL_NAME,
89         temperature=0,
90         api_key=api_key,
91     )
92     llm_with_tools = llm.bind_tools(TOOLS)
93
94     messages = [
95         SystemMessage(
96             content=(
97                 "Tu es un agent d'assistance pour etudiant. "
98                 "Tu peux utiliser des outils pour calculer, chercher dans des notes locales "
99                 "et preparer un plan de revision. "
100                 "Utilise un outil seulement si c'est utile. "
101                 "Quand un outil est utilise, base ta reponse finale sur son resultat."
102             )
103         ),
104         HumanMessage(content=question),
105     ]
106
107     first_response = llm_with_tools.invoke(messages)
108     messages.append(first_response)
109
110     if first_response.tool_calls:
111         for tool_call in first_response.tool_calls:
112             tool_name = tool_call["name"]
113             tool_args = tool_call["args"]
114             selected_tool = TOOLS_BY_NAME[tool_name]
115             tool_output = selected_tool.invoke(tool_args)
116
117             messages.append(
118                 ToolMessage(
119                     content=tool_output,
120                     tool_call_id=tool_call["id"],
121                 )
122             )
123
124             final_response = llm_with_tools.invoke(messages)
125             return final_response.content
126
127     return first_response.content
128
129
130 def main():
131     print("Agent d'etude pret.")
132     print("Exemples :")
133     print("- Calcule la moyenne de [12, 15, 9].")
134     print("- Cherche le mot RAG dans mes notes.")
135     print("- Prepare-moi un plan de revision sur les agents IA.")
136     print("Tapez 'quit' pour quitter.\n")
137
138     while True:
139         question = input("Question > ").strip()
140
141         if not question:
142             print("Veuillez saisir une question.\n")
143             continue
```

```
144
145     if question.lower() in {"quit", "exit", "q"}:
146         print("Fin du programme.")
147         break
148
149     try:
150         answer = run_agent(question)
151         print("\n--- Reponse de l'agent ---")
152         print(answer)
153         print()
154     except Exception as exc:
155         print(f"\nErreur : {exc}\n")
156
157
158 if __name__ == "__main__":
159     main()
```

9. Lecture du code pas à pas

9.1. Les trois outils

Le script définit trois outils :

1. **calculer_moyenne** : calcule moyenne, min et max.
2. **chercher_dans_les_notes** : cherche un mot-clé dans un fichier local.
3. **generer_plan_revision** : produit un plan simple et structuré.

Pourquoi ces trois outils ?

- le premier montre un besoin de **calcul** ;
- le deuxième montre un besoin de **lecture locale** ;
- le troisième montre un besoin de **production utile**.

9.2. Le décorateur @tool

Le décorateur **@tool** transforme une fonction Python en outil utilisable par le modèle.

Idée importante : le modèle ne voit pas seulement du texte. Il voit aussi une description structurée des outils disponibles.

9.3. Le modèle Groq

Le script utilise ChatGroq.

Rôle du modèle :

- comprendre la demande ;
- décider si un outil est nécessaire ;
- choisir le bon outil ;
- produire une réponse finale.

9.4. bind_tools

La ligne avec `llm.bind_tools(TOOLS)` relie les outils au modèle.

Conséquence : le modèle peut décider d'appeler un outil s'il juge que c'est nécessaire.

9.5. La logique agentique

Le script suit une logique simple :

1. l'utilisateur pose une question ;

2. le modèle analyse cette question ;
3. s'il faut un outil, il le demande ;
4. Python exécute l'outil ;
5. le résultat revient au modèle ;
6. le modèle produit la réponse finale.

C'est exactement ce que l'on veut faire comprendre dans ce TP : un agent n'est pas seulement un modèle ; c'est un **système outillé**.

10. Exécution du programme

Lancer le script avec :

```
1 python agent_etudiant_tp.py
```

Exemples de questions à tester :

- Calcule la moyenne de [12, 15, 9].
- Cherche le mot RAG dans mes notes.
- Prepare-moi un plan de revision sur les agents IA.

11. Activités pratiques

Activité 1 : identifier l'outil choisi

Tester trois questions différentes :

- une question de calcul ;
- une question de recherche dans les notes ;
- une question de planification.

Question 1 :

Outil probablement utilisé :

Question 2 :

Outil probablement utilisé :

Question 3 :

Outil probablement utilisé :

Activité 2 : comparer chatbot et agent

Poser cette question :

Calcule la moyenne de [13, 16, 9, 14] puis explique si le resultat est satisfaisant.

Question : en quoi cette situation est-elle plus agentique qu'un chatbot classique ?

Activité 3 : modifier les notes locales

Ouvrir le fichier `notes_agent_tp.txt` et ajouter 3 nouvelles lignes.

Exemple :

- une ligne sur MCP ;
- une ligne sur OpenClaw ;
- une ligne sur n8n.

Puis tester :

Cherche le mot MCP dans mes notes.

Observation :

Activité 4 : rendre l'agent plus utile

Modifier le texte de consigne système dans le script pour que l'agent :

- réponde de manière plus courte ;
- ou plus pédagogique ;
- ou sous forme de liste en 3 points.

Nouvelle consigne système :

Effet observé :

Activité 5 : ajouter un quatrième outil

C'est l'activité à plus forte valeur formatrice du TP.

Ajouter un nouvel outil au choix, par exemple :

- `lister_les_mots_cles()` ;
- `compter_les_lignes_des_notes()` ;
- `donner_definition_simple(mot)` ;
- `preparer_quiz(sujet)`.

Compétence visée : apprendre à **concevoir un outil pertinent** puis à l'ajouter à un agent.

Nom du nouvel outil :

Rôle de cet outil :

Question de test :

Résultat obtenu :

12. Questions de compréhension

1. Pourquoi un agent est-il plus qu'un simple chatbot ?
2. Quel est le rôle d'un outil dans ce TP ?
3. Pourquoi le fichier local de notes est-il intéressant ?
4. Pourquoi le calcul de moyenne est-il un bon exemple d'outil ?
5. À quoi sert `bind_tools` ?
6. Quel est l'intérêt pédagogique d'ajouter un quatrième outil ?

13. Livrable final

Le livrable final doit contenir :

- une capture du script exécuté ;
- au moins 3 questions testées ;
- les réponses obtenues ;
- une explication de l'outil utilisé dans chaque cas ;
- une modification du fichier de notes ;
- l'ajout d'un quatrième outil personnel ;
- une conclusion personnelle de 8 à 12 lignes.

14. Grille d'évaluation

Critère	Barème	Note
Compréhension du fonctionnement global d'un agent	4 pts	
Qualité des tests et des observations	4 pts	
Compréhension des outils et de leur rôle	4 pts	
Qualité de la personnalisation du système	4 pts	
Ajout pertinent d'un quatrième outil	4 pts	
Total	20 pts	

15. Ce que l'étudiant gagne vraiment avec ce TP

Ce TP donne une compétence simple mais très utile :

savoir concevoir un premier agent outillé et l'étendre par soi-même.

Cette compétence peut ensuite être réutilisée pour :

- un assistant personnel ;
- un agent de support ;
- un agent de révision ;
- un agent connecté à des documents ou à des APIs.

16. Ressources utiles

- Groq Console : <https://console.groq.com>
- LangChain : <https://python.langchain.com>

Idée à retenir : la vraie compétence n'est pas seulement d'utiliser un modèle, mais de concevoir un système utile autour du modèle.